Prompt & Command Set Classifier

Jonathan Stokes

November 3, 2026

---

1 Introduction

As a Linux user I sometimes find myself running command line interface (CLI) commands that I do not fully understand. Even if I look up the documentation for the commands it is not always clear what effect a command a will have on my specific machine. One alternative to parsing the documentation to understand what a set of commands does is to run the commands in a sandbox see what happens. While running commands in a sandbox simple in theory, practically I ran into a problems since it can be impossible to verify whether a set of commands produced the desired outcome without side effects.

Consider the following questions that are among the 100 most viewed questions on askubuntu.com:

1. How do I save terminal output to a file?
2. How to list all installed packages?
3. How do I install Ubuntu alongside a pre-installed Windows with UEFI?

If we have access to a Ubuntu sandbox, can we verify CLI commands answering these questions? "How do I save the terminal output to a file?" can be verified by writing the terminal output to a file and checking that the files contents matches the terminal output. But it is not clear how to verify a set of commands that purport to list all installed packages since it would require knowing all packages installed in the sandbox. And no set of CLI commands run in a Ubuntu sandbox can satisfy the question "How do I install Ubuntu alongside a pre-installed Windows with UEFI?". This inability to verify every command set a problem for any sandbox verification system, not just for Linux CLI command verification system.

This project attempts to solve this problem by classifying questions/prompts and CLI command sets proposed by LLMs to answer these questions. The following section introduces a simple 8bit Adder system and example prompts which are easier to reason about than Ubuntu. I then outline 5 categories that I hope cover all possible system prompts and show that the 8but Adder prompts can be classified with a decider model like Cloudflare's Clef-Flash model. Finally I return to the three Linux prompts and show that a similar Clef-Flash decider model system can also be used to classify Linux prompts.

2 8bit Adder System

Lets imagine a simple 8bit adder service. The aim of this system is to be easy to reason about rather than useful. The 8bit Adders functionality is detailed below:

1. The adder is a micro service whose endpoints are accessible via routes of the form GET `http://localhost/endpoint`
2. The adder supports adding 8 bit non-negative numbers A and B at endpoint GET `http://localhost/add/{A}/{B}` and returns A+B = R if R is 8 bit or R=InvalidRequest if it is not.
3. The adder also supports getting the adder logs at endpoint GET `http://localhost/logs` which return all requests made to the service and the services responses as a json in the form request_id, timestamp, request, response.
4. All other requests to the service for the form `http://localhost/X` where X is any string including the empty string but excepting the routes detailed above return R=InvalidRequest.

3 8bit Adder Prompt Taxonomy

The prompt taxonomy I settled on for system prompts is:

- **Unknowable:** Prompts that cannot be answered by the system alone now.
- **Unverifiable:** Prompts and command sets that cannot be verified by the prompter.
- **Unverifiable Minimal:** Prompts and command sets that cannot be verified by the prompter and whose system commands are the minimal set required.
- **Verifiable:** Prompts and command sets that can be verified by the prompter.
- **Verifiable Minimal:** Prompts and command sets that can be verified by the prompter and whose system commands are the minimal set required.

Below are examples 8bit adder prompts, Muse Code generated commands for the 8bit adder system, and my classification of the prompts and command sets:

- How do I tell what language the system is written?
  - Commands: GET `http://localhost/language`
  - Classification: Unknowable
- How do I know the services log timestamps are accurate?
  - Commands: GET `http://localhost/add/10/20`, GET `http://localhost/time`, GET `http://localhost/logs`
  - Classification: Unverifiable
- How do I know the services log timestamps are accurate?
  - Commands: GET `http://localhost/timestamps`
  - Classification: Unverifiable Minimal
- How do I add 5 and 7 using the adder service?
  - Commands: GET `http://localhost/add/5/7`, GET `http://localhost/logs`
  - Classification: Verifiable
- How do I add 5 and 7 using the adder service?

– Commands: GET `http://localhost/add/5/7`

– Classification: Verifiable Minimal

Note that these examples are drawn from a synthetic dataset and to get Muse Code to reliably generate non-minimal command sets for testing I asked that it generate commands that would self verify the result.

4 8bit Adder Prompt Classification

The prompt classifier queries the Cloudflare decider model Clef-Flash with either 1 or 3 of the questions below following the question tree in Figure 1.

- Q1: Is the prompt answerable by the system according to its specification?
- Q2: Does the prompt ask the system to perform an action with a checkable result, and not about things that require perfect knowledge of the system (such as whether the system logs are complete or accurate, what other users did, timestamp accuracy, etc.)?
- Q3, Q4: Given the system specification is every request in the LLM_response required for the prompt's task?

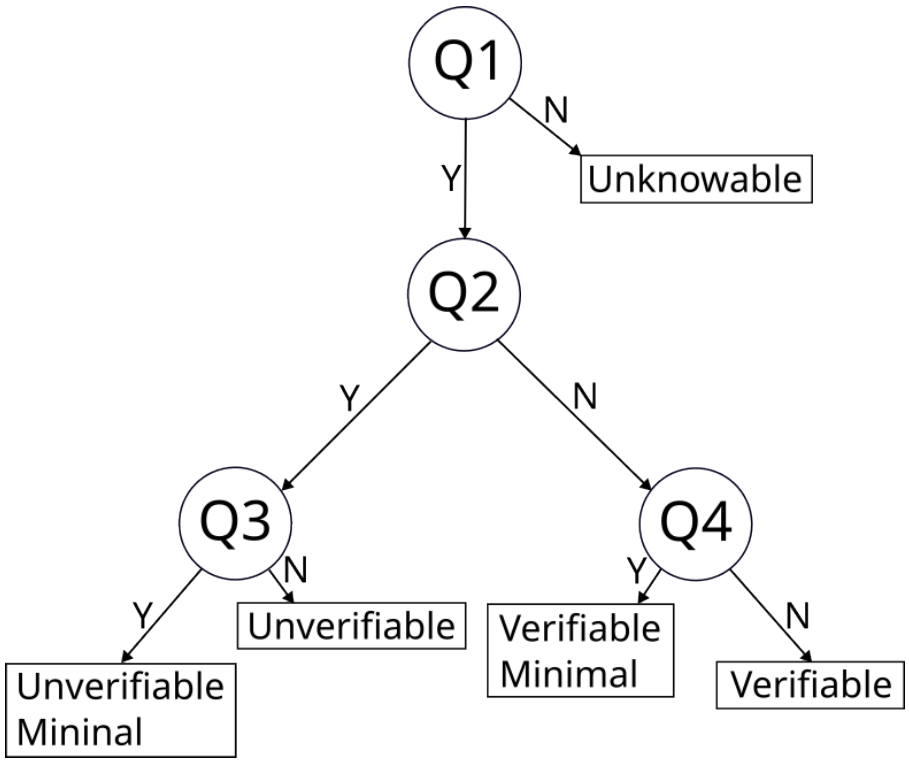


Figure 1

Clef-Flash model returns a binary answer and a confidence probability for the answer. With Muse Code's help I tuned both the questions and the decision cutoff points for the answer confidence in order to achieve the following results. I think this tuning likely over fit the 8bit Adder prompt data set which consists of only 50 prompts, 10 in each category. However my goal here was not to have this prompt classification framework generalize but only to show that it can work.

On the set of 8bit Adder prompts command set pairs in adder_gt.csv, using the question tree in Figure 1, the questions above, and the cutoff points {q1:0.07, q2:0.82, q3:0.3, q4:0.6}; the Clef-Flash model achieved a categorization accuracy of 40/50 (80.0%). The per category accuracy, precision, and recall are given in Table 1.

Category	**Accuracy**	**Precision**	**Recall**
Unknowable	10/10 (100.0%)	0.62	1.00
Unverifiable	9/10 (90.0%)	0.90	0.90
Unverifiable Minimal	6/10 (60.0%)	0.86	0.60
Verifiable	8/10 (80.0%)	1.00	0.80
Verifiable Minimal	7/10 (70.0%)	0.78	0.70

Table 1: 8Bit Adder Prompt & Command Set Classification Results

The confusion matrix for the prompt and command set classification of the 8bit Adder prompts is given in Figure 2.

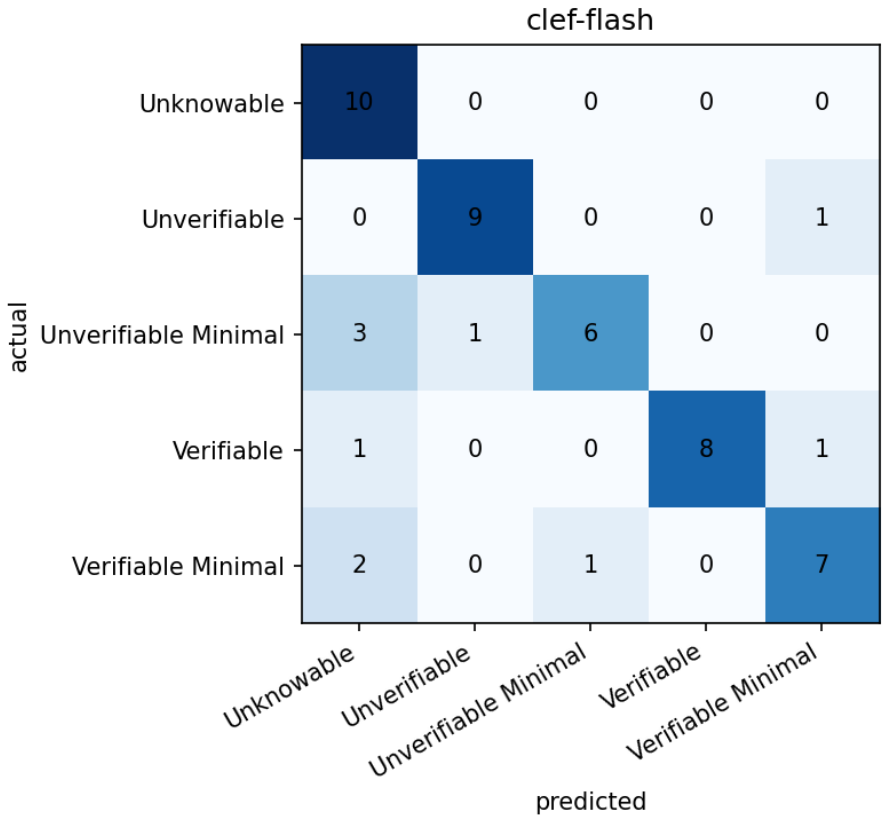


Figure 2: 8Bit Adder Prompt & Command Set Classification Confusion Matrix

5 Ubuntu Prompt Classification

Given that the classification results for the 8bit Adder prompts and command sets appeared to work I decided to see if the the classification would also work on the 100 most viewed questions on askubuntu.com. For the prompts I used the question titles as the questions themselves can be long and unfocused. I used Muse Code to get command sets satisfying the prompts and then I labeled the prompt command set pairs to produce the data set in linux_question_responses_groundtruth.csv. The fact that I labeled the Linux prompt is a weakness because my labeling might be incorrect. But for a POC I think self labeling the ground truth is acceptable and the following results do suggest that classifying Linux prompts using a

decide model like Clef-Flash is possible even if not all the ground truth classifications are correct.

As for the 8bit Adder I used Muse Code to tune the probability cutoffs and the phrasing of question Q2. The wording of Q2 depends on what actions the system takes which can and cannot be verified. In the future I would like to find a phrasing of Q2 that generalizes better across systems. The system definition, Q2 phrasing, and the cutoffs I settled on where:

- System Definition: The system is a Linux OS Ubuntu distribution version 26.04.
- Q2: Does the prompt ask the system to perform an action with a checkable result (modifying an application, file, or user), and not for information only the system itself knows (package list, system version, users, logs, etc)?
- Cutoffs: {q1:0.21, q2:0.11, q3:0.08, q4:0.25}

Using these parameters the categorization accuracy of Linux prompts and command set pairs was 79/100 (79.0%). While this is impressive at first glance the high degree of accuracy is largely due to most of the prompts in the ground truth set being classified as Verifiable Minimal. The per category accuracy, precision, recall, and confusion matrix in Table 2 and Figure 3 paint a more accurate picture of the mixed results.

Category	**Accuracy**	**Precision**	**Recall**
Unknowable	1/5 (20.0%)	0.20	0.20
Unverifiable	0/0 (n/a)	0.00	n/a
Unverifiable Minimal	12/14 (85.7%)	0.60	0.86
Verifiable	0/1 (0.0%)	0.00	0.00
Verifiable Minimal	66/80 (82.5%)	0.96	0.82

Table 2: Linux Prompt & Command Set Classification Results

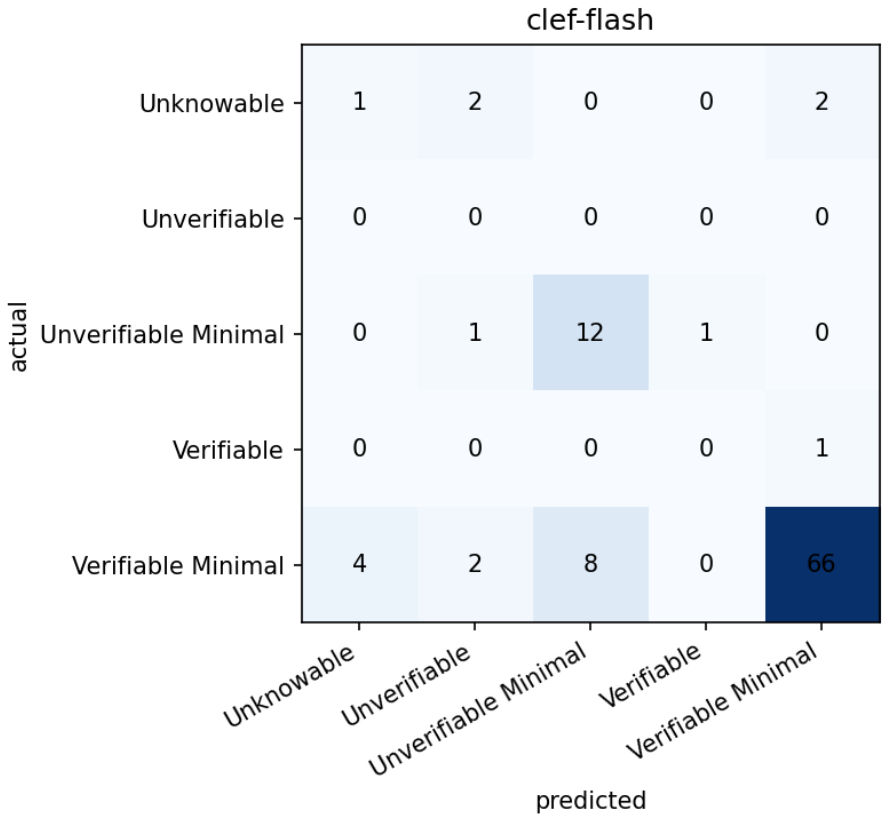


Figure 3: Linux Prompt & Command Set Classification Confusion Matrix

Perhaps a more useful classification given that most questions on askubuntu.com and the set of commands pairs generated by Muse Code are Verifiable Minimal is the binary classification as Verifiable Minimal or not. This binary classification has some justification in that Verifiable Minimal command sets can be verified in a sandbox while other classifications cannot. The accuracy, precision, recall, and specificity of the Verifiable Minimal binary classifier for Linux prompts and command set pairs using the Clef-Flash model are:

- Accuracy: 83.0%
- Precision: 0.96
- Recall: 0.82
- Specificity: 0.85

Statistics which are computed from 66 True Positives, 3 False Positives, 14 False Negatives, and 17 True Negatives.

6 Conclusion

My take way from the results above are that while I need a bigger data set and a more rigorous means of labeling it, Linux prompt command set pairs probably can be classified as Verifiable Minimal or not, especially if a decider model is tuned for this classification. Whether a useful automated sandbox system can be built to validate Linux commands using these classifications to flag prompt command set pairs that cannot be verified remains to be seen.